

Loci· 本地、私密、 可版本管理

把你的文档库装进 LLM 里——不联网、不上传、不花钱，检索走 BM25+向量混合+精排，生成走本地 Ollama。

四章·十九页

01

CHAPTER

项目背景

目标、意义、上下文与项目定位。从「为什么做」出发。

P. 03 – 06 · 3 pages

02

CHAPTER

系统设计

技术栈、架构图、RAG 检索流水线、持久化与前后端协议。

P. 07 – 11 · 5 pages

03

CHAPTER

实施评估

能力矩阵、性能指标、测试结果与版本演进数据。

P. 12 – 15 · 4 pages

04

CHAPTER

反思总结

踩过的坑、关键经验、未来改进与项目总结。

P. 16 – 19 · 4 pages

01

项目背景

从「为什么做」出发——**本地知识库 RAG** 解决了什么痛点？它的目标、意义与上下文又是什么？本章回答这三个根本问题。

“把文档库装进 *LLM*，不联网、不上传、不花钱。

三个核心目标

从大模型应用的「本地化、可用性、可演进」三个维度出发，明确 Loci 的设计目标。

01

PRIVACY · 隐私

完全本地化部署

0 API Key、0 网络请求。文档与对话数据全部留在本机，LLM 走本地 Ollama，向量库走本地 FAISS。

// hf_hub_offline = 1

02

CAPABILITY · 能力

高精度混合检索

BM25 关键词 + FAISS 向量 RRF 融合 + CrossEncoder 精排，三阶段流水线兼顾语义泛化与专有名词命中。

// BM25:0.6 + Vector:0.4

03

EVOLVABILITY · 演进

可版本管理的产品

向量库版本快照、一键回滚、6 个独立存储类（chat / favorites / tags / settings）、解耦的模块边界。

// keep_last = 10 versions

为什么需要本地RAG？

云端 RAG 服务存在隐私、成本、网络依赖三大硬伤，Loci 用「本地化 + 混合检索 + 智能 Fallback」给出回应。



从 *v1.0* 到 *v5.1* 的演进

Loci 起始于 NLP 课程实验 3（LangChain + FAISS + Ollama + Gradio），经过 4 个月 5 个大版本迭代，演化为生产级本地 RAG 系统。



// POSITION

生产级 本地 *RAG* 系统

- NLP 课程实验 3 成果
- 面试导向 LLM 项目
- 5 Tab 全功能产品
- 19 页 PPT 完整呈现

02

系统设计

从**技术栈选型**到**架构分层**，从**RAG 检索流水线**到**数据持久化**——本章拆解 Loci 的工程实现。

“*BM25 + 向量 RRF + CrossEncoder 精排，是 Loci 检索精度的三板斧。*”

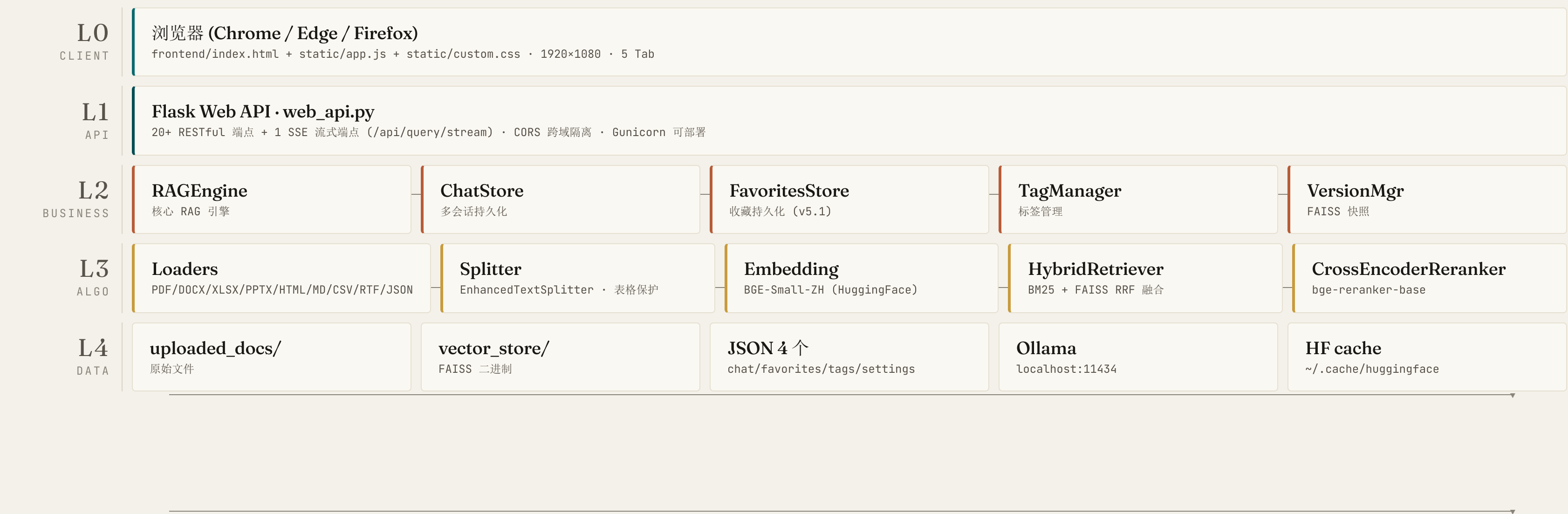
5层技术栈

从用户界面到磁盘存储，LocI 在每一层都选用「本地优先、轻依赖、社区活跃」的开源方案。

L1	前端呈现层 原生 HTML + CSS 变量主题 + Vanilla JS · SSE <code>fetch().body.getReader()</code> · 字体 <code>Fraunces / Manrope / JetBrains Mono</code>	UI · STATE
L2	后端服务层 Flask 3 + <code>Flask-CORS</code> · RESTful + <code>SSE</code> · CORS 跨域隔离 · 20+ HTTP 端点	API · ROUTING
L3	RAG 框架层 LangChain 1.x <code>LCEL</code> 链式调用 · PromptTemplate + StrOutputParser · FAISS 本地向量库 · BM25Retriever	RETRIEVAL · GENERATION
L4	算法模型层 Embedding <code>BGE-Small-ZH</code> (<code>HuggingFace</code> 离线) · Rerank <code>bge-reranker-base</code> (<code>CrossEncoder</code>) · LLM <code>deepseek-r1:7b</code> (<code>Ollama</code>)	ENCODE · RANK · GENERATE
L5	数据持久化层 JSON + <code>threading.Lock</code> 原子写 · FAISS <code>.faiss/.pkl</code> 二进制 · Ollama <code>localhost:11434</code> · HF cache <code>~/.cache/huggingface</code>	PERSIST · CACHE

4 层架构+1 个客户端

从浏览器到磁盘，每一层职责清晰、依赖单向收敛——`config` 是叶子节点，被所有模块依赖；`web_api` 是唯一串联所有模块的入口。



6 步流水线 + 智能 Fallback

从用户问题到最终答案，Loci 走完「混合检索 → 精排 → 阈值过滤 → Fallback 决策 → LLM 生成 → 流式输出」6 个阶段。



// Fallback

RAG Fallback 智能兜底

当知识库无相关答案时，自动切到纯 LLM 路径，用黄色提示条告知用户「答案未参考文档」。

RAG_FALLBACK_ENABLED = True

4 类 *存储* + 4 个共同特征

4 个独立 Store 类遵循统一的「JSON + lock + 原子写 + 容错读」风格，5.0 重构后全部归位 `data/` 目录。

STORE	文件	LOCK	ATOMIC	FALLBACK
CHAT ChatStore	chat_history.json	✓	✓	✓
TAG TagManager	tags.json	✓	✓	✓
FAV FavoritesStore (v5.1)	favorites.json	✓	✓	✓
CFG SettingsManager	user_settings.json	✓	✓	✓
VS VersionManager	vector_store_versions/	•	rename	—

// SHARED PATTERN

JSON Store 的 4 个共同特征

01 顶层是 dict
{"items": [...]} · 方便后续扩展字段

02 写入 temp + rename
atomic 原子操作 · 避免半写状态

03 JSON 损坏容错读
返回空结构 · 不抛异常

04 threading.Lock
写操作加锁 · 避免并发竞争

03

实施 *评估*

从**能力矩阵**到**性能指标**，从**测试结果**到**版本演进数据**——本章用数据回答「做得怎么样」。

“数据不撒谎：26 个测试、15 种格式、4 层架构、5 个版本。

7大能力·5 Tab 全覆盖

从智能问答到版本快照，Loci 围绕 7 个核心能力构建产品。覆盖文档、检索、生成、管理、诊断全链路。



智能问答

多轮上下文注入 · SSE 流式输出 · 来源追溯 · RAG Fallback 兜底 · 自动会话管理

5 轮

MAX HISTORY TURNS

TAB 1



混合检索

BM25 + 向量 RRF 融合

40%

TOP5 HIT RATE LIFT

RETRIEVAL



知识库

15 种格式 · 标签管理 · 全文搜索 · URL 摄取 · 文件夹监控

15

FORMATS SUPPORTED

TAB 2



收藏管理

问答收藏 · 关键词高亮 · Markdown / JSON 导出

TAB 3



多会话

多会话持久化 · 一键删除 · 历史回放

TAB 4



设置中心

引擎诊断 · 主题切换 · 危险操作

TAB 5



版本快照

FAISS 索引快照 · 一键回滚

10

VERSIONS KEPT

SETTINGS

4 大 *KPI* + 测试结果

从检索精度到响应延迟，从测试通过率到端到端验证，Loci 用 4 个核心 KPI + 26 个单元测试 + Playwright E2E 验证工程完成度。

+40%

RETRIEVAL HIT RATE

v3.0 引入混合检索后，Top5 命中率相对 v2.1 提升 40%。

5-8_x

QUERY SPEEDUP

Rerank 精排后上下文缩短，检索链路整体提速 5-8 倍。

26_{/26}

UNIT TESTS PASS

RAG Fallback / Settings / Exporters 三套单测，100% 通过。

0_{API}

NETWORK CALLS

HF_HUB_OFFLINE=1 · 无任何外部网络请求 · 完全离线运行。

单元测试

// UNIT TESTS · PYTEST

✓ test_rag_fallback.py · 6 个 RAG 阈值决策分支

✓ test_settings_manager.py · 10 个 settings 读写场景

✓ test_exporters.py · 10 个 Markdown / PDF 导出测试

✓ 回归测试通过率 26 / 26 (100%)

✓ 关键 Bug: RAG_FALLBACK_THRESHOLD 阈值未生效已修复

E2E 验证

// PLAYWRIGHT · END-TO-END

✓ 5 Tab 加载 & 切换正常

✓ 设置页开关 & 持久化生效

✓ 收藏导出 Markdown / PDF 200

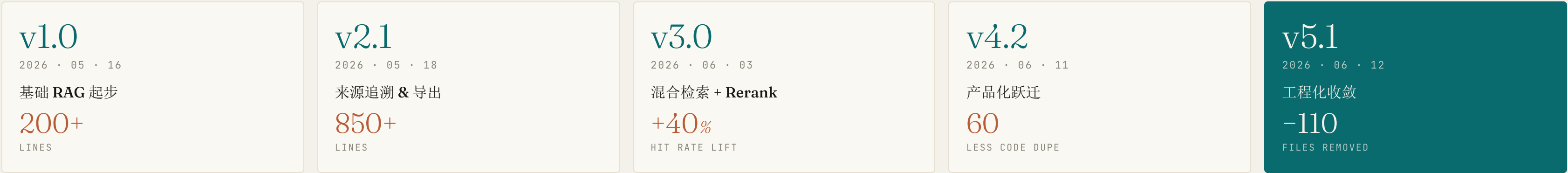
✓ 智能问答 Fallback 触发正常

✓ 「重新生成」按钮实际调用 SSE

✓ 多会话持久化 & 切换无错乱

5 个大版本 · 4 周迭代

从 2026-05-16 的 200 行 v1.0 到 2026-06-12 的 v5.1，4 周内完成 5 个大版本迭代，删除 110+ 文件、收敛到 4 层目录架构。



04

反思总结

从**经验教训**到**挑战与解法**，从**未来改进**到**项目总结**——本章回顾 Loci 的得失，并为下一阶段指明方向。

“ 一个项目的真正价值，不在于它能跑多快，而在于它让你看到多深。”

踩过的4个坑

每个 Bug 都是一次对框架/工具理解的深化——这 4 个 v4.x 关键 Bug 的修复经历，是 Loci 工程化路上最值得分享的部分。

01 LANGCHAIN 1.X COMPAT

BM25Retriever API 变更

LangChain 1.x 移除了 `get_relevant_documents()`，必须改用 `invoke()`。混用新旧 API 会导致 `TypeError`，但堆栈信息往往指向错误位置，调试困难。

```
# 修复: retriever.invoke(query) 替代 retriever.get_relevant_documents(query)
```

02 BGE SIGMOID THRESHOLD

RAG Fallback 阈值未生效

v4.1 配置了 `RAG_FALLBACK_THRESHOLD=0.3`，但 `_resolve_sources()` 缺少判定逻辑，导致「数据库设计」问题检索到不相关 CIFAR-10 文档仍强行生成答案。

```
// 修复: 阈值 0.3→0.5 (BGE 官方建议) + _resolve_sources() 加判定
```

03 STATEFUL SSE

「重新生成」按钮死代码

v4.0 「重新生成」按钮只刷新 UI，没有真正调用后端的 `askQuestion({regenerate:true})`。需先在 `web_api.py` 端增加 `regenerate` 字段透传到 `rag_engine`，再在前端真实发起 SSE 调用。

```
// 修复: 前端实际 SSE 调用 + 后端 query() 接受 regenerate 参数
```

04 PATH POLLUTION

doc.name 路径污染

v2.x FAISS 存储 `doc.name` 时混入相对路径 (`./uploaded_docs\file.pdf`)，API 透传到前端后路径显示混乱。需在 `load_documents` 阶段统一用 `Path(p).name` 提取文件名。

```
// 修复: doc.metadata["source"] = Path(file_path).name
```

演进 *路线* · v6.0 → v9.0

v5.1 是「能用」的下一步是「好用、易扩展、可协作」。以下 4 个版本的演进路线图，回答「LocI 还能怎么走」。

v5.1	当前·工程化收敛 4 层目录隔离 · 移除 Gradio 入口 · 删除 110+ 非必要文件	2026 · 06 · 12
v6.0	React/Vue 前端 + SQLite 替代 JSON 10w+ 收藏/会话后 JSON 读写会变慢，SQLite 提升 IO。React/Vue 接管前端，状态管理更可维护。 FRONTENDSQLITE	Q3 · 2026
v7.0	多用户隔离 + 权限管理 基于 user_id 隔离聊天/收藏/标签/会话。引入 JWT 鉴权 + RBAC 权限模型，支持团队协作。 AUTHMULTI-USER	Q4 · 2026
v8.0	WebSocket 双向通信 + Agent 框架 SSE 升级为 WebSocket，支持取消生成、追问。引入 LangGraph 让 LLM 自调用工具链。 WEBSOCKETAGENTLANGGRAPH	Q1 · 2027
v9.0	微服务拆分 + Milvus 分布式向量库 前端/后端/算法 3 个独立服务。Milvus 替代 FAISS，支持亿级文档水平扩展。 MICROSERVICEMILVUS	Future

— END OF PRESENTATION

感谢 倾听

Loci 是一个 *持续演进* 的项目——欢迎反馈、issue、PR，与我一起把它做得更好。

Q&A 欢迎提问·讨论·建议

// AUTHOR

炳材 · 广东东软学院
人工智能专业 · 大三
2027 届 · Trae IDE 重度用户

// OPEN SOURCE

LangChain · FAISS · Ollama
Flask · BGE · rank-bm25
jieba · trafilatura · watchdog

// CONTACT

GitHub: bingcai-lz / loci
Course: NLP 实验 3
Started: 2026 · 05 · 16